

Tecnología Digital VI

Redes Neuronales Convolucionales

Licenciatura en Tecnología Digital - UTDT

Hasta acá

En las anteriores clases vimos varios conceptos tanto teóricos como prácticos de Redes Neuronales.

Se presenta como un formalismo muy poderoso pero por el momento no queda clara la diferencia con los métodos de Machine Learning clásico.

Nuestro argumento (en la primera parte de la materia) para decir que la Redes Neuronales eran prometedoras era que iban a dejar de tratar a las columnas como features completamente independientes sin relación semántica.

En lo visto hasta acá, ¿qué hace de distinto una red feed-forward fully-connected con respecto a este tema? ¿por qué deberían andar mejor que un XGBoost en problemas tabulares?

Hasta acá



La respuesta podría ser “nada” o “no mucho” y sería correcto.

El formalismo crudo por sí solo no le da un tratamiento muy diferente a la interacción entre las columnas que lo que proponían otros métodos como Random Forest o XGBoost.

Es decir, proponen un entrenamiento desde un formalismo matemático distinto pero no hay ningún esfuerzo particular en aprovechar la semántica que puede tener la interacción de ciertos features.

A partir de acá

Desde esta clase la idea va a ser empezar a explotar de manera particular estas relaciones para entender cómo y por qué las Redes Neuronales son tan performantes en contextos particulares como el procesamiento de texto, imágenes o audio.

Vamos a comenzar con imágenes, en donde nuestros features son los píxeles. ¿Podemos tratar a los píxeles como features completamente independientes entre sí? Podemos (de hecho ya lo hicimos), pero buscar explotar la Localización Espacial nos va a llevar por mucho mejor camino.

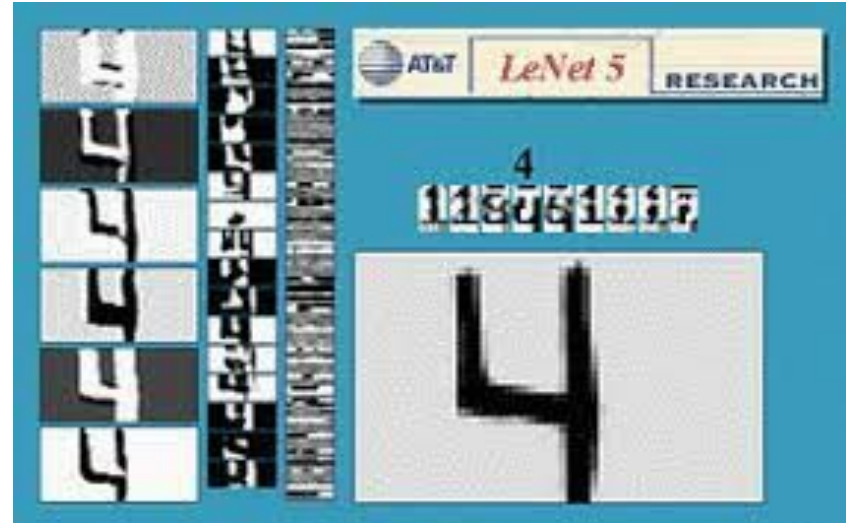
Imágenes

Cuando trabajamos con MNIST, simplemente levantamos las imágenes y las aplanamos para tener 784 features por cada muestra.

Usar esos 784 valores para entrenar un Random Forest o una NN fully-connected no tiene un diferencial conceptual muy grande.

```
import pandas as pd
data = pd.read_csv('./digit-recognizer/train.csv')
data.head()
```

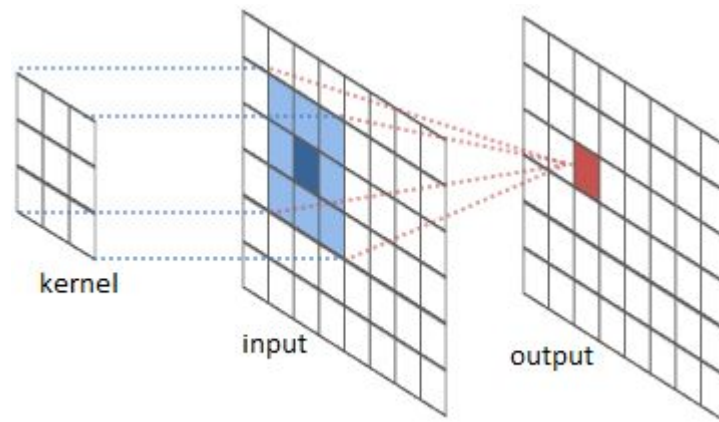
	label	pixel0	pixel1	pixel2	pixel3	pixel4	pixel5	pixel6	pixel7	pixel8	...	pixel774	pixel775	pixel776	pixel777	pixel778
0	1	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0
2	1	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0
3	4	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0



Convoluciones

Una convolución es una operación matemática que toma dos señales (audio, imágenes, etc) y produce una tercera señal.

En general una de las dos señales es una imagen completa, mientras que la otra es un pequeño kernel que se va deslizando y aplicando sobre la original.



Convolución 1-D continua

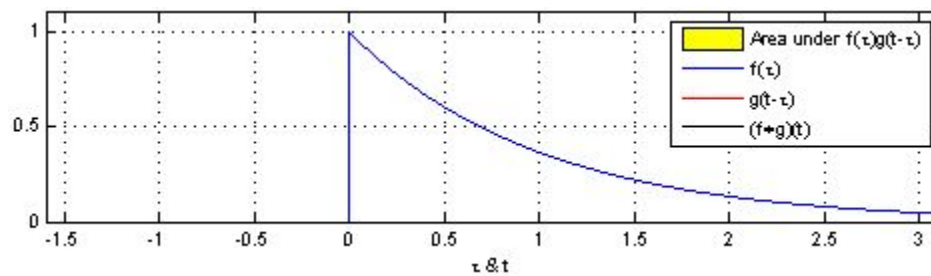
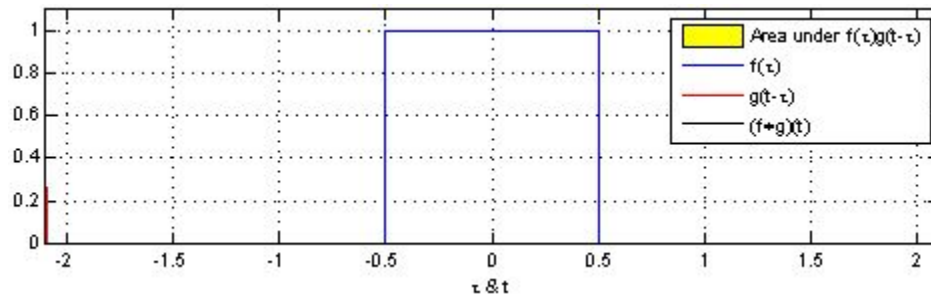
Para entender qué es una convolución, podemos comenzar con la definición *más pura* al aplicar a dos funciones de una sola dimensión y continuas.

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau)g(t - \tau) d\tau$$

Coloquialmente:

- Para un t en particular, espejamos la g (de $-\infty$ a ∞) y la desplazamos según t .
- Multiplicamos punto a punto la f y esta nueva g espejada y desplazada.
- *Sumamos* todo. (En 1-D continuo es lo mismo que calcular el área bajo la curva de la multiplicación).

Convolución 1-D continua



Convolución 2-D discreta

La versión discreta 2-D cambia un índice por dos y la integral por una sumatoria.

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(m, n) K(i - m, j - n)$$

Coloquialmente:

- Calculamos cada píxel de salida (i, j) por separado.
- Multiplicamos píxel a píxel la imagen original y la segunda imagen espejada.
- Sumamos todos los valores.

Convolución 2-D discreta en la práctica

En general, la I suele ser una imagen *full-size*. La K suele ser una pequeña “imagen” de, por ejemplo, 3×3 denominado *Kernel*.

Entonces, en el caso de un kernel de 3×3 , la operación se resume a ir desplazando el kernel por la I , haciendo el cálculo de multiplicar píxel a píxel y acumular los valores.

A la salida se la denomina *feature map*.

Convoluciones

¿Qué pasa si aplico los siguientes kernels a una imagen?

$1/9$	$1/9$	$1/9$
$1/9$	$1/9$	$1/9$
$1/9$	$1/9$	$1/9$

-1	-1	-1
0	0	0
1	1	1

Convoluciones

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img_gris = cv2.imread('utdt.jpg', cv2.IMREAD_GRAYSCALE)

imagen_blur = cv2.blur(img_gris, (3, 3))

kernel_prewitt_y = np.array([
    [-1, -1, -1],
    [0, 0, 0],
    [1, 1, 1]
])

gradiente_y = cv2.filter2D(img_gris, -1, kernel_prewitt_y)

fig, axs = plt.subplots(1, 3, figsize=(18, 6))
```

```
axs[0].imshow(img_gris, cmap='gray')
axs[0].set_title('Imagen Original')
axs[0].axis('off')

axs[1].imshow(imagen_blur, cmap='gray')
axs[1].set_title('Desenfoque (Blur) con OpenCV')
axs[1].axis('off')

axs[2].imshow(gradiente_y, cmap='gray')
axs[2].set_title('Filtro de Prewitt Horizontal con OpenCV')
axs[2].axis('off')

plt.tight_layout()
plt.savefig('resultados_opencv.png')
plt.show()
```

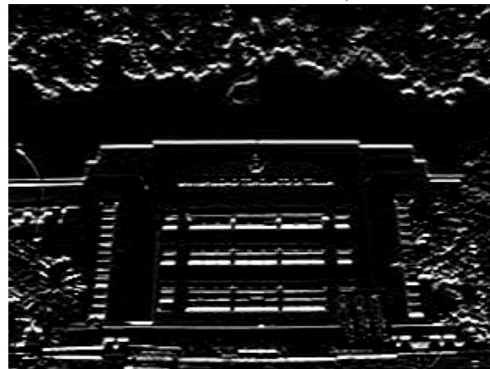
Imagen Original



Desenfoque (Blur) con OpenCV



Filtro de Prewitt Horizontal con OpenCV



Capa convolucional

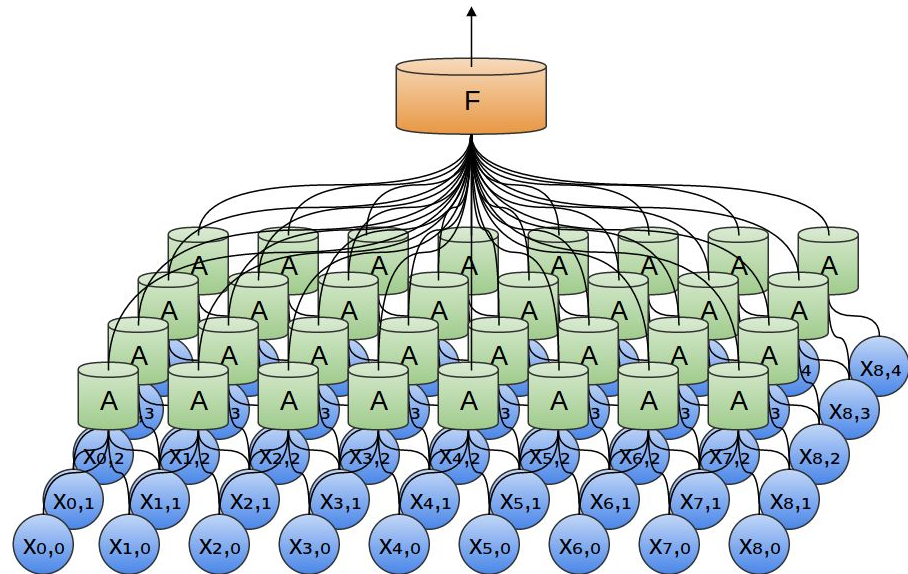
¿Cómo usamos esta noción de convolución en Redes Neuronales?

Podemos crear una capa en donde las entradas no estén *fully-connected* sino que solamente permitamos conectarse a los píxeles que compartan una cierta vecindad (por ejemplo 3x3) para que la multiplicación por los pesos de la red sea análogo a aplicar un filtro por convolución.

¿No podían hacer esto las capas fully-connected? Sí, pero tenían que aprenderlo y eso puede resultar muy costoso.

Capa convolucional

En una capa convolucional, no tenemos todas las conexiones como en una capa fully connected. Las conexiones representan una pequeña convolución para cada patch. ¿Qué numeritos les damos a cada una de estas operaciones?

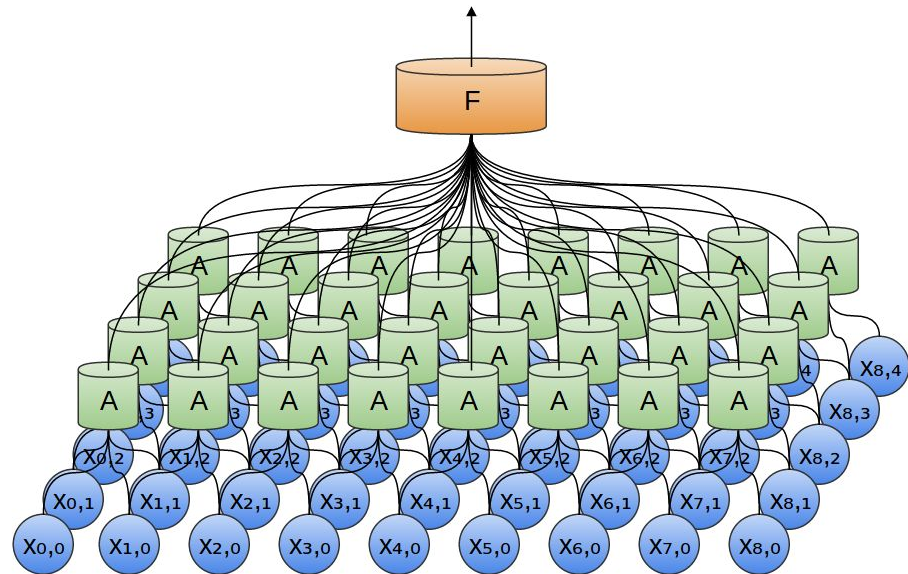


Capa convolucional

En una capa convolucional, no tenemos todas las conexiones como en una capa fully connected. Las conexiones representan una pequeña convolución para cada patch. ¿Qué numeritos les damos a cada una de estas operaciones?

Ninguno!

Es justamente lo que la red debe aprender.

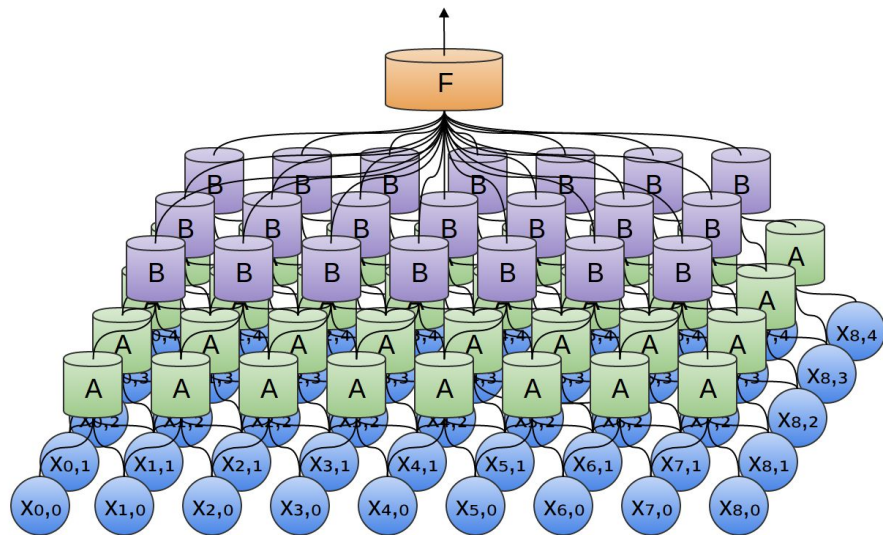


Red convolucional

Para a partir de capas convolucionales armar una red convolucional, podemos simplemente ir agregando capa tras capa como hicimos en los ejemplos fully connected.

Hay varias cosas que hay que definir:

- ¿Quién rompe la linealidad?
- ¿Qué tamaño tienen las convoluciones?
- Los *patches*, ¿se solapan? ¿cuánto?
- ¿Qué hago en los bordes de la imagen?



Si paso un filtro (hago la convolución) de 3x3 a una imagen de 5x5. ¿De qué tamaño queda la imagen resultante (o feature map)?

Si paso un filtro (hago la convolución) de 3×3 a una imagen de 5×5 . ¿De qué tamaño queda la imagen resultante (o feature map)?

Así como lo definimos, nos queda una imagen de 3×3 .

Esto nos trae dos problemas:

- La imagen se achica: por pasar un filtro, se achica el tamaño de la imagen. No suele ser algo deseado.
- Los bordes participan menos. Este, en problemas de imágenes *naturales*, es mucho menos importante que el anterior.

Padding

El *padding* se refiere al tratamiento que le damos a las bordes al aplicar las convoluciones. Una opción simple puede ser rellenando con 0 un borde adicional a la imagen original para que el tamaño de la imagen resultante quede invariante.

¿Es *inocuo* rellenar con 0? No, cualquier decisión que se tome puede tener un efecto no deseado según el tipo de características que el filtro está tratando e resaltar.

Tipos de Padding:

- ¿Qué rellenar?
 - Valid Padding o $P=0$: Sin padding, el filtro se aplica donde entra completo. Se achica la salida.
 - Same Padding o $P=1$: Se agrega lo necesario para que la imagen no cambie de tamaño.
 - Hay otros tipos de padding como Full Padding que no se utilizan.
 - Las capas conv2d de Pytorch tienen un parámetro padding para manejar esto (por default igual a 0).
- ¿Cómo rellenar?
 - Zero Padding: rellenamos con 0. Genera un marco artificial.
 - Replication Padding: repite el valor del borde.
 - El parámetro de PyTorch es padding_mode y por default es *zeros*.

En la definición de convolución que dimos vamos desplazando el kernel *de 1 en 1*, pero esto no tiene por qué ser así.

El *stride* es el *tranco* que da el filtro cada vez que se mueve.

- Nos movemos muy poco? Feature map más grande, píxeles afectados por más de un filtro.
- Nos movemos mucho? Feature map más chico, píxeles en un solo filtro o incluso en ninguno.

¿Cuál situación es preferible? Depende.

Stride

En la práctica se pueden usar diferentes tipos de stride pero hay que tener en cuenta que este es el parámetro que mayor impacto tiene en el tamaño de la salida. Cuanto más saltemos, más abruptamente se achica la imagen.

Históricamente el stride se solía utilizar en 1 y el efecto de achicar la imagen se dejaba para otra componente (Pooling), pero con el tiempo se empezaron a utilizar otros valores de stride porque, a diferencia del pooling, este parámetro genera un achicamiento *aprendible*.

En PyTorch las capas convolucionales tienen un parámetro *stride* que por default es 1.

Tamaño de la salida

Las diferentes decisiones como tamaño, padding y stride tienen un impacto directo en el tamaño de la salida.

Asumiendo imágenes y filtros *cuadrados*, el tamaño del output lo podemos caracterizar de la siguiente manera:

$$O = \frac{I - K + 2P}{S} + 1$$

No-Linealidad

La convolución se presenta como una nueva función matemática, pero en el caso discreto termina siendo nuevamente una combinación lineal de los pesos y los inputs.

Si solamente ponemos capas convolucionales puras una detrás de la otra, volvemos nuevamente al problema de que componer transformaciones lineales sigue siendo una transformación lineal.

¿Qué rompe la linealidad en las CNNs?

- Las capas de pooling, no es su objetivo principal pero igualmente rompen la linealidad.
- Las funciones de activaciones, siguen estando presentes como en las primeras redes.

Pooling

Supongamos que tengo una serie de filtros que detecta algo en particular como un ojo. Muchas veces no nos interesa exacto saber dónde está el ojo sino que ya es muy útil poder reconocer “por esta zona hay un ojo”.

Las capas de *pooling* resumen la información de un grupo de píxeles vecinos. De esta manera los filtros ganan robustez ante la traslación (no importa si el ojo está un poco corrido en la siguiente imagen). Además, funciona como una reducción de la dimensionalidad para ganar en espacio y cómputo.

Por último, estas capas aumentan el *campo receptivo* así que los sucesivos filtros trabajan sobre sectores más grandes de la imagen.

Tipos de Pooling

Las formas de *resumir la información* mediante *pooling* pueden ser varias:

- Max Pooling: Se queda con el máximo de todos los valores del parche.
- Average Pooling: Promedia todos los valores.
- Min Pooling: Se queda con el mínimo.
- Global Average Pooling: Un caso especial de Average Pooling con utilidad en ciertas redes.

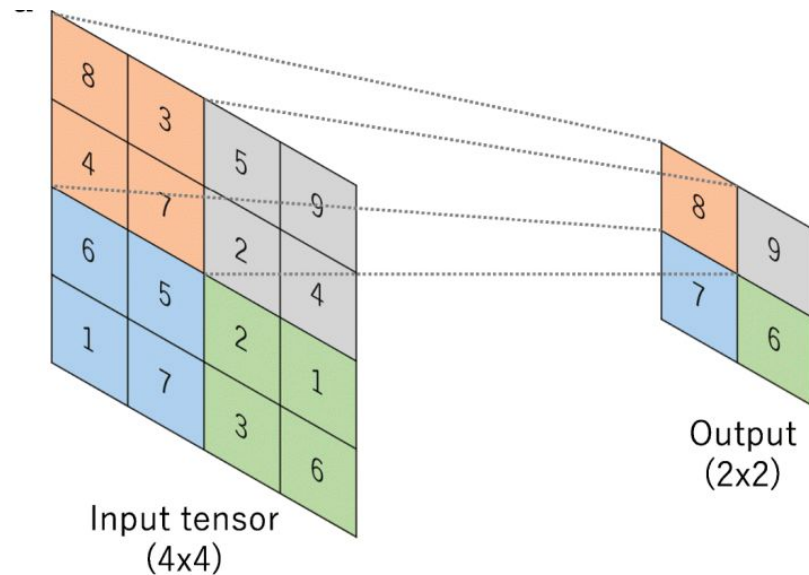
En todos los casos, no hay nada a *aprender*. No son capas que tengan pesos a ajustar sino que simplemente realizan una operación mecánica “De este parche de 2x2 píxeles, me quedo con el más grande”.

Max Pooling

Max Pooling es de lo más utilizados en las CNNs.

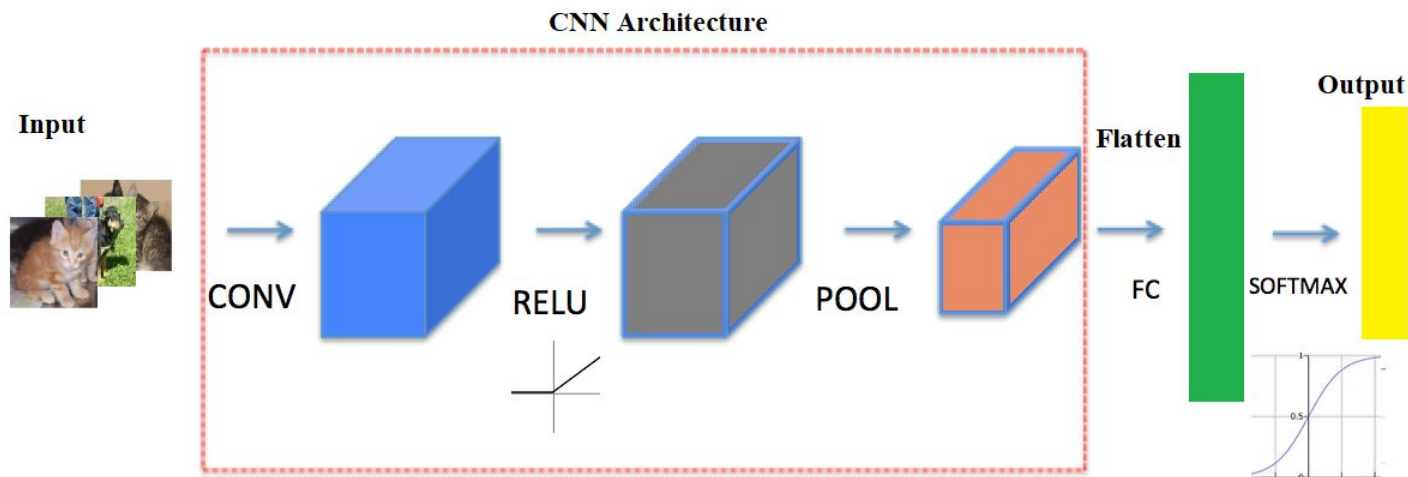
En general, la aplicación de filtros se traduce en detección de diferentes patrones u objetos (“encontré un borde vertical”, “encontré un ojo”).

El Max Pooling resalta justamente el mayor valor, conservando la detección del patrón en todo el parche.



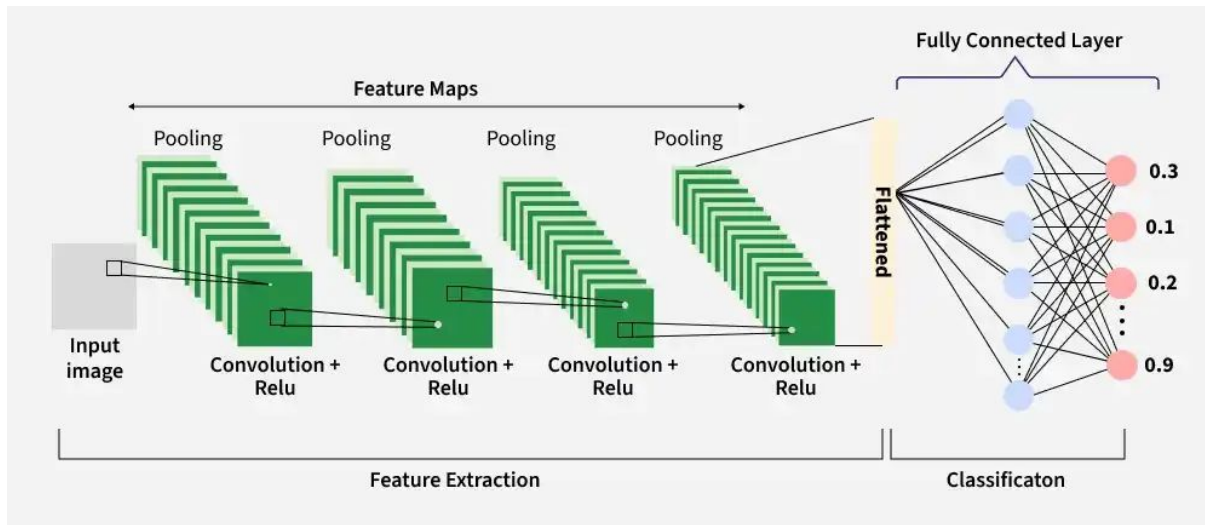
Bloque CNN

Con todo lo visto, podemos definir los *bloques convolucionales*. Estos bloques pueden variar en las diferentes aplicaciones pero en general se suele asociar una capa convolucional con una función de activación, y luego algún tipo de pooling.



CNN Completa

Las CNNs son redes que combinan muchos de estos bloques con otras componentes. Están pensadas para representar la extracción de features visuales de la imágenes. En general, suele tener adosado al final una serie de capas que “toma la decisión” (clasificación o regresión) en base a los features extraídos.



Vamos a hacer nuestra primera red convolucional *from scratch* para el problema de MNIST.

```
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader
import time

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

df = pd.read_csv('./digit-recognizer/train.csv')

etiquetas = torch.tensor(df.iloc[:, 0].values, dtype=torch.long)
pixeles = df.iloc[:, 1:].values / 255.0 # Normalizamos de 0 a 1

imagenes_2d = torch.tensor(pixeles, dtype=torch.float32).view(-1, 1, 28, 28)

dataset_completo = TensorDataset(imagenes_2d, etiquetas)

train_size = int(0.8 * len(dataset_completo))
val_size = len(dataset_completo) - train_size
train_db, val_db = torch.utils.data.random_split(dataset_completo, [train_size, val_size])

train_loader = DataLoader(train_db, batch_size=64, shuffle=True)
val_loader = DataLoader(val_db, batch_size=64, shuffle=False)

print(f"Total imágenes: {len(dataset_completo)}")
print(f"Train: {len(train_db)} | Validation: {len(val_db)}")

Total imágenes: 42000
Train: 33600 | Validation: 8400
```

Nuestra red van a ser dos bloques convolucionales uno detrás del otro, y al finalizar una capa *fully-connected* para mapear los features obtenidos a las 10 clases posibles.

```
modelo_mnist = nn.Sequential(  
    nn.Conv2d(1, 16, kernel_size=3, padding=1),  
    nn.ReLU(),  
    nn.MaxPool2d(kernel_size=2),  
  
    nn.Conv2d(16, 32, kernel_size=3, padding=1),  
    nn.ReLU(),  
    nn.MaxPool2d(kernel_size=2),  
  
    nn.Flatten(),  
  
    nn.Linear(32 * 7 * 7, 128),  
    nn.ReLU(),  
    nn.Linear(128, 10)  
) .to(device)  
  
criterio = nn.CrossEntropyLoss()  
optimizador = optim.Adam(modelo_mnist.parameters(), lr=0.001)
```



```
print("Iniciando entrenamiento y validación...")
inicio = time.time()

for epoca in range(10):
    modelo_mnist.train()
    correctos_train, total_train = 0, 0
    perdida_train_acum = 0.0

    for imagenes, etiquetas in train_loader:
        imagenes, etiquetas = imagenes.to(device), etiquetas.to(device)

        optimizador.zero_grad()
        salidas = modelo_mnist(imagenes)
        perdida = criterio(salidas, etiquetas)
        perdida.backward()
        optimizador.step()

        perdida_train_acum += perdida.item()
        _, prediccion = torch.max(salidas, 1)
        total_train += etiquetas.size(0)
        correctos_train += (prediccion == etiquetas).sum().item()

    modelo_mnist.eval()
    correctos_val, total_val = 0, 0
    perdida_val_acum = 0.0

    with torch.no_grad():
        for imagenes, etiquetas in val_loader:
            imagenes, etiquetas = imagenes.to(device), etiquetas.to(device)

            salidas = modelo_mnist(imagenes)
            perdida = criterio(salidas, etiquetas)

            perdida_val_acum += perdida.item()
            _, prediccion = torch.max(salidas, 1)
            total_val += etiquetas.size(0)
            correctos_val += (prediccion == etiquetas).sum().item()

    print(f"\n--- Época {epoca+1} ---")
    print(f"TRAIN -> Pérdida: {perdida_train_acum/len(train_loader):.4f} | Precisión: {100 * correctos_train/total_train:.2f}%")
    print(f"VAL -> Pérdida: {perdida_val_acum/len(val_loader):.4f} | Precisión: {100 * correctos_val/total_val:.2f}%")

print(f"\nEntrenamiento listo en {time.time() - inicio:.2f} segundos.")
```

Iniciando entrenamiento y validación...

```
--- Época 1 ---
TRAIN -> Pérdida: 0.3235 | Precisión: 90.27%
VAL -> Pérdida: 0.0950 | Precisión: 97.14%
```

```
--- Época 2 ---
TRAIN -> Pérdida: 0.0847 | Precisión: 97.51%
VAL -> Pérdida: 0.0620 | Precisión: 97.98%
```

```
--- Época 3 ---
TRAIN -> Pérdida: 0.0545 | Precisión: 98.31%
VAL -> Pérdida: 0.0572 | Precisión: 98.27%
```

```
--- Época 4 ---
TRAIN -> Pérdida: 0.0411 | Precisión: 98.74%
VAL -> Pérdida: 0.0460 | Precisión: 98.54%
```

```
--- Época 5 ---
TRAIN -> Pérdida: 0.0337 | Precisión: 98.84%
VAL -> Pérdida: 0.0425 | Precisión: 98.77%
```

```
--- Época 6 ---
TRAIN -> Pérdida: 0.0262 | Precisión: 99.12%
VAL -> Pérdida: 0.0514 | Precisión: 98.40%
```

```
--- Época 7 ---
TRAIN -> Pérdida: 0.0202 | Precisión: 99.35%
VAL -> Pérdida: 0.0452 | Precisión: 98.71%
```

```
--- Época 8 ---
TRAIN -> Pérdida: 0.0152 | Precisión: 99.50%
VAL -> Pérdida: 0.0461 | Precisión: 98.73%
```

```
--- Época 9 ---
TRAIN -> Pérdida: 0.0138 | Precisión: 99.57%
VAL -> Pérdida: 0.0393 | Precisión: 98.95%
```

```
--- Época 10 ---
TRAIN -> Pérdida: 0.0121 | Precisión: 99.55%
VAL -> Pérdida: 0.0590 | Precisión: 98.50%
```

Entrenamiento listo en 14.60 segundos.

CNN from scratch

Las imágenes de MNIST son muy chiquitas, conocidas, en escala de grises y de un contexto muy particular.

¿Se acuerdan de Chihuahuas o Muffins de arándanos?



Chihuahuas

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
from PIL import Image
import time
import urllib.request
import matplotlib.pyplot as plt

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

transformaciones = transforms.Compose([
    transforms.Resize((128, 128)),
    transforms.ToTensor(),
])

dataset_completo = datasets.ImageFolder(root='./chihuahua/train/', transform=transformaciones)

train_size = int(0.8 * len(dataset_completo))
val_size = len(dataset_completo) - train_size
train_db, val_db = torch.utils.data.random_split(dataset_completo, [train_size, val_size])

train_loader = DataLoader(train_db, batch_size=32, shuffle=True)
val_loader = DataLoader(val_db, batch_size=32, shuffle=False)

print(f"Clases detectadas automáticamente: {dataset_completo.classes}")
print(f"Imágenes para entrenar: {len(train_db)} | Para validar: {len(val_db)}")

Clases detectadas automáticamente: ['chihuahua', 'muffin']
Imágenes para entrenar: 3786 | Para validar: 947
```

```
modelo_binario = nn.Sequential(
    nn.Conv2d(3, 32, kernel_size=3, padding=1),
    nn.ReLU(),
    nn.MaxPool2d(2),

    nn.Conv2d(32, 64, kernel_size=3, padding=1),
    nn.ReLU(),
    nn.MaxPool2d(2),

    nn.Conv2d(64, 128, kernel_size=3, padding=1),
    nn.ReLU(),
    nn.MaxPool2d(2),

    nn.Flatten(),

    nn.Linear(128 * 16 * 16, 256),
    nn.ReLU(),
    nn.Dropout(0.5),
    nn.Linear(256, 2)
).to(device)

criterio = nn.CrossEntropyLoss()
optimizador = optim.Adam(modelo_binario.parameters(), lr=0.0005)
```

Chihuahuas

```
print("Iniciando entrenamiento y validación...")
inicio = time.time()
epocas = 10

for epoca in range(epocas):
    modelo_binario.train()
    correctos_train, total_train = 0, 0
    perdida_train_acum = 0.0

    for imagenes, etiquetas in train_loader:
        imagenes, etiquetas = imagenes.to(device), etiquetas.to(device)

        optimizador.zero_grad()
        salidas = modelo_binario(imagenes)
        perdida = criterio(salidas, etiquetas)
        perdida.backward()
        optimizador.step()

        perdida_train_acum += perdida.item()
        _, prediccion = torch.max(salidas, 1)
        total_train += etiquetas.size(0)
        correctos_train += (prediccion == etiquetas).sum().item()

    modelo_binario.eval()
    correctos_val, total_val = 0, 0
    perdida_val_acum = 0.0

    with torch.no_grad():
        for imagenes, etiquetas in val_loader:
            imagenes, etiquetas = imagenes.to(device), etiquetas.to(device)

            salidas = modelo_binario(imagenes)
            perdida = criterio(salidas, etiquetas)

            perdida_val_acum += perdida.item()
            _, prediccion = torch.max(salidas, 1)
            total_val += etiquetas.size(0)
            correctos_val += (prediccion == etiquetas).sum().item()

    print(f"Época {epoca+1}/{epocas}")
    print(f"  TRAIN -> Loss: {perdida_train_acum/len(train_loader):.4f} | Acc: {100 * correctos_train/total_train:.2f}%")
    print(f"  VAL   -> Loss: {perdida_val_acum/len(val_loader):.4f} | Acc: {100 * correctos_val/total_val:.2f}%")

print(f"\nEntrenamiento completado en {time.time() - inicio:.2f} segundos.")
```

Iniciando entrenamiento y validación...

Época 1/10

TRAIN -> Loss: 0.5482 | Acc: 71.32%

VAL -> Loss: 0.4213 | Acc: 79.09%

Época 2/10

TRAIN -> Loss: 0.3744 | Acc: 84.36%

VAL -> Loss: 0.3434 | Acc: 85.53%

Época 3/10

TRAIN -> Loss: 0.3021 | Acc: 87.51%

VAL -> Loss: 0.4128 | Acc: 82.89%

Época 4/10

TRAIN -> Loss: 0.2527 | Acc: 89.73%

VAL -> Loss: 0.2759 | Acc: 88.81%

Época 5/10

TRAIN -> Loss: 0.2177 | Acc: 91.10%

VAL -> Loss: 0.2760 | Acc: 88.17%

Época 6/10

TRAIN -> Loss: 0.1742 | Acc: 93.19%

VAL -> Loss: 0.2929 | Acc: 88.70%

Época 7/10

TRAIN -> Loss: 0.1498 | Acc: 94.16%

VAL -> Loss: 0.2914 | Acc: 88.81%

Época 8/10

TRAIN -> Loss: 0.1328 | Acc: 94.98%

VAL -> Loss: 0.2783 | Acc: 89.44%

Época 9/10

TRAIN -> Loss: 0.0875 | Acc: 96.72%

VAL -> Loss: 0.3221 | Acc: 89.02%

Época 10/10

TRAIN -> Loss: 0.0790 | Acc: 97.23%

VAL -> Loss: 0.2932 | Acc: 89.97%

Entrenamiento completado en 309.54 segundos.

Con una red entrenada desde cero, pudimos reconocer bien 9 de cada 10 imágenes con un entrenamiento de un par de minutos.

Si bien es un resultado bastante bueno, no se acerca a cuando con mucho menos entrenamiento pudimos separar perros de gatos en la primera clase casi con 100% de accuracy.

¿Qué cambió? En ese momento no utilizamos una red convolucional desde cero, sino que utilizamos una *preentrenada* que ya tenía mucha información útil.